

BUILD THIS COMPLETE APPLICATION

You are an expert senior mobile engineer, computer vision engineer, AI engineer, UI/UX designer, and software architect.

I want you to build a complete mobile application called:

"PosePilot"

AI Real-Time Photography Pose Coach

The core purpose of this application is to solve a common photography problem:

When someone wants to take a photo, the person standing in front of the camera often does not know how to pose. They usually search Pinterest, Instagram, Google Images, or other photos for pose references. This is inconvenient, and a reference image does not actively guide the person while they are standing in front of the camera.

This application solves that problem by turning the phone camera into a REAL-TIME AI POSE COACH.

The photographer opens the app, points the camera at the person, selects a pose or reference photo, and the application analyzes the person's body in real time and tells them exactly how to move.

Examples:

- "Raise your right hand"
- "Lower your left hand"
- "Move slightly to the right"
- "Move slightly backward"
- "Turn your face left"
- "Turn your face right"
- "Straighten your shoulders"
- "Move your left foot outward"
- "Keep your face straight"
- "Look at the camera"
- "Tilt your chin slightly down"
- "Put your left hand in your pocket"
- "Move closer to the camera"

The application should visually show arrows and guidance over the camera preview.

The final goal is:

CAMERA → DETECT PERSON → UNDERSTAND POSE → COMPARE TO TARGET → GIVE REAL-TIME INSTRUCTIONS → PERSON CORRECTS POSE → POSE MATCHES → CAPTURE PHOTO

IMPORTANT PRODUCT VISION

Do NOT build this as a simple pose gallery.

The primary innovation is REAL-TIME ACTIVE POSE GUIDANCE.

A normal pose app says:

"Try this pose."

Our application should say:

"Raise your right hand ↑ "

then:

"Move slightly left ←"

then:

"Turn your face slightly right ↷"

then:

"Perfect pose ✓"

and finally:

"3... 2... 1... Capture"

The user should feel as though there is a personal photographer/pose coach inside the phone.

MAIN APPLICATION MODES

Build the application around these modes.

MODE 1 — AI LIVE POSE COACH

This is the main feature.

Open camera.

Detect the person's body.

Display body landmarks/skeleton.

Select a target pose.

Compare the user's current pose with the target.

Display correction instructions.

Examples:

- Raise right hand
- Lower left hand
- Turn head left
- Turn head right
- Straighten shoulders
- Move left
- Move right
- Move closer
- Move backward
- Separate feet
- Bring feet closer
- Bend elbow
- Straighten elbow
- Look at camera
- Tilt chin down/up

When the user reaches the target:

"POSE READY ✓"

Optionally:

3...2...1...

Capture automatically.

MODE 2 — POSE LIBRARY

Create a pose library.

Categories:

Casual

- Standing
- Hands in pocket
- Leaning
- Looking away
- Crossed arms
- Walking

Instagram / Social

- Street
- Cafe
- Fashion
- Travel
- Night
- Lifestyle

Couple

- Standing together
- Walking together
- Looking at each other
- Shoulder pose
- One behind another
- Casual couple pose

Group

- 2 people
- 3 people
- 4 people
- 5+ people
- Friends
- College group
- Graduation

Formal

- LinkedIn

- Interview
- Corporate
- Graduation
- Portrait

Travel

- Landmark
- Scenic
- Walking
- Sitting
- Adventure

Allow the user to select a category and a target pose.

MODE 3 — REFERENCE PHOTO MODE

This is a very important feature.

The user can upload a photo from:

- Gallery
- File picker
- Camera

The application analyzes the reference image.

Extract the person's pose/body landmarks.

Convert the reference pose into a target pose representation.

Then open the live camera.

The application compares the live person's body to the reference pose.

Example:

REFERENCE PHOTO ↓ POSE EXTRACTION ↓ TARGET LANDMARKS / ANGLES ↓ LIVE CAMERA ↓ CURRENT LANDMARKS ↓ POSE DIFFERENCE ↓ CORRECTIONS ↓ MATCH

The application should tell the person how to recreate the reference pose.

Example:

"Raise right elbow"

"Turn your face slightly left"

"Move your left foot outward"

"Move slightly right"

"Perfect — reference pose matched"

This feature directly replaces the need to keep looking at Pinterest while posing.

MODE 4 — PHOTOGRAPHER MODE

The photographer is holding the phone.

The subject stands in front of the camera.

The photographer should be able to manually give instructions too.

Provide buttons such as:

- Raise Left Hand
- Raise Right Hand
- Move Left
- Move Right
- Move Forward
- Move Back
- Turn Left
- Turn Right
- Straighten
- Look at Camera
- Look Away
- Hands in Pocket
- Smile
- Relax

AI guidance and manual guidance should work together.

MODE 5 — GROUP POSE MODE

Support multiple people.

Detect multiple people in the camera frame.

For each detected person, calculate:

- Position
- Body orientation
- Body alignment
- Relative spacing

Provide group guidance.

Examples:

"Person 1 move left"

"Person 2 move slightly backward"

"Person 3 raise right hand"

"Move closer together"

"Spread out slightly"

"Center the group"

The goal is to make group photos look more organized.

MODE 6 — PHOTO QUALITY CHECK

After capturing a photo, analyze it.

Check:

- Blur
- Face visibility
- Eyes open if detectable
- Person fully inside frame
- Body cut-off
- Composition
- Centering
- Lighting condition if possible

- Pose match
- Multiple-person alignment

Show a simple report.

Example:

PHOTO ANALYSIS

Pose Match ✓ Face Visibility ✓ Framing ✓ Body Position ✓ Blur ⚠ Lighting ✓

If a problem is detected:

"Try again — slight motion blur detected."

UI / UX REQUIREMENTS

The UI should be modern, clean, minimal, professional and visually attractive.

Do not make it look like a basic college project.

Use a camera-first interface.

Suggested home page:

POSEPILOT

Your AI Photography Coach

[AI POSE COACH]

[REFERENCE PHOTO]

[POSE LIBRARY]

[MY PHOTOS]

On the live camera screen:

LIVE CAMERA UI

Top:

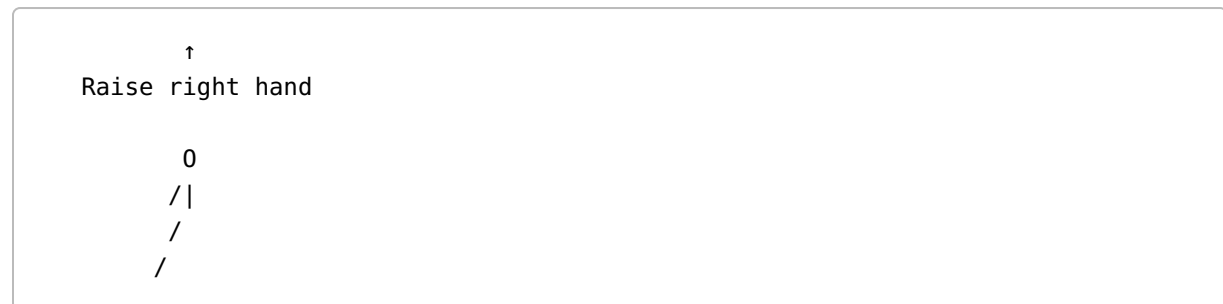


Camera preview in the center.

Draw body skeleton/landmarks over the detected person.

Show guidance visually.

Example:



Bottom:

Pose Match: 87%

"Raise your right arm"

Buttons:

[RETAKE] [CAPTURE]

Use smooth animations.

Instructions should update in real time.

VISUAL GUIDANCE

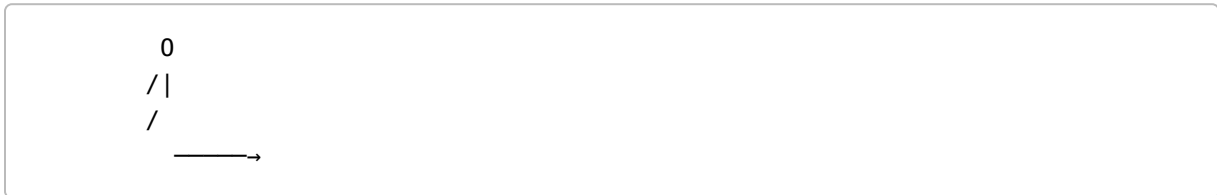
This is extremely important.

Do not only display text.

Draw visual instructions directly over the camera feed.

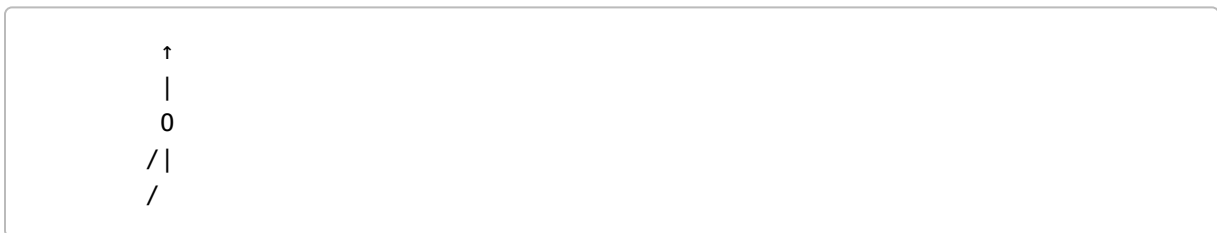
Examples:

Move Right



"Move right"

Raise Hand



"Raise right hand"

Turn Face



"Turn your face left"

Move Feet



"Move your feet closer"

Use arrows, circles, lines, highlighting and directional indicators.

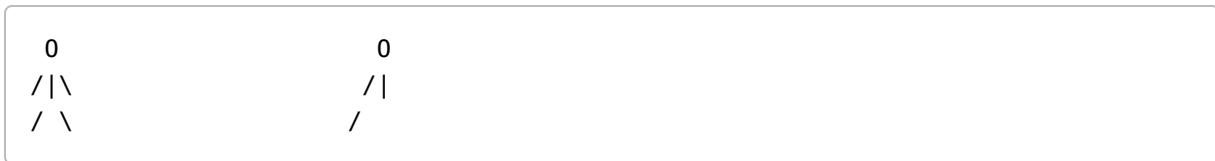
TARGET POSE / GHOST POSE

Add a target-pose visualization.

The target pose should be displayed as a semi-transparent skeleton or ghost silhouette.

For example:

LIVE PERSON TARGET



The user should move their body until their pose aligns with the target.

Display:

Pose Match: 64%

Pose Match: 82%

Pose Match: 95%

TARGET REACHED ✓

Use visual feedback.

AI / COMPUTER VISION ARCHITECTURE

Do NOT attempt to train a huge AI model for the MVP.

Use existing on-device pose estimation technology.

Preferred options:

1. Google ML Kit Pose Detection
2. MediaPipe Pose / Pose Landmarker

For Android MVP, prefer:

Kotlin Android Studio Jetpack Compose CameraX Google ML Kit Pose Detection

The core processing should preferably happen on-device.

The application should work with minimal or no internet connection for real-time pose coaching.

POSE LANDMARKS

Use body landmarks such as:

- Nose
- Left eye
- Right eye
- Left ear
- Right ear
- Left shoulder
- Right shoulder
- Left elbow
- Right elbow
- Left wrist
- Right wrist
- Left hip
- Right hip
- Left knee
- Right knee
- Left ankle
- Right ankle
- Feet

Use these landmarks to calculate body geometry.

POSE ANALYSIS ENGINE

Create a dedicated Pose Analysis Engine.

Do not put all logic inside UI code.

Create clean modules/classes.

Suggested architecture:

Camera Layer ↓ Pose Detection Layer ↓ Landmark Processing Layer ↓ Pose Analysis Engine ↓ Target Pose Engine ↓ Correction Engine ↓ Guidance Engine ↓ UI Overlay

POSE MATHEMATICS

For MVP use geometry and rules.

Do not use AI classification for everything.

Calculate:

- Joint angles
- Shoulder angle
- Hip angle
- Elbow angle
- Knee angle
- Head orientation
- Relative landmark distances
- Person position in frame
- Body orientation

For example:

Shoulder → Elbow → Wrist

Calculate the elbow angle.

If:

$\text{angle} > \text{target} + \text{tolerance}$

then:

"Bend your elbow slightly"

If:

$\text{angle} < \text{target} - \text{tolerance}$

then:

"Straighten your elbow"

Otherwise:

"√"

TARGET POSE REPRESENTATION

Create a reusable target-pose data structure.

Example concept:

PoseTemplate

{ id, name, category, description, targetLandmarks, targetAngles, targetPositions, tolerance, instructions }

Each pose should contain the information necessary to compare the user's pose.

RULE ENGINE

Create a dedicated rule engine.

Example conceptual structure:

PoseRule

{ targetBodyPart, targetValue, tolerance, correctionDirection, instructionText, priority }

Examples:

Rule: Right elbow target angle = 90° Tolerance = $\pm 15^\circ$

Result:

Current angle = 130° → "Bend your right arm"

Current angle = 80° → "Almost perfect"

Current angle = 92° → "Perfect"

PRIORITY SYSTEM

Do NOT show 10 instructions simultaneously.

Only show the most important corrections.

For example:

Priority 1: Person not centered.

Instruction: "Move right"

Then:

Priority 2: Hand position incorrect.

Instruction: "Raise right hand"

Then:

Priority 3: Head angle incorrect.

Instruction: "Turn face left"

Then:

"POSE READY ✓"

This should make the experience simple and understandable.

POSE MATCH SCORE

Create a pose matching score.

Example:

Head alignment 95% Shoulders 90% Left arm 82% Right arm 94% Hip 97% Legs 89%

Overall:

Pose Match = 91%

Use a weighted score.

Do not make the score meaningless.

It should correspond to actual landmark/angle differences.

VOICE ASSISTANT

Add optional text-to-speech.

Examples:

"Raise your right hand."

"Move slightly left."

"Turn your face to the right."

"Hold."

"Perfect."

"Three."

"Two."

"One."

"Capture."

Add a mute button.

Do not repeatedly speak the same instruction every frame.

Add cooldown/debouncing.

PERFORMANCE REQUIREMENTS

This is a real-time application.

Optimize it.

Requirements:

- Camera preview should be smooth.
 - Pose detection should be performed efficiently.
 - Do not process every frame if unnecessary.
 - Drop stale frames.
 - Always prioritize the latest camera frame.
 - Avoid memory leaks.
 - Avoid unnecessary allocations.
 - Do not run heavy processing on the UI thread.
 - Keep guidance responsive.
 - Keep overlay rendering smooth.
-

PRIVACY

Pose processing should preferably happen on-device.

Do not upload camera frames to a server for normal pose coaching.

Do not store photos unless the user explicitly chooses to save them.

Clearly request camera and gallery permissions only when needed.

OFFLINE FIRST

The following should work without internet:

- Camera
- Pose detection
- Pose comparison
- Basic pose library
- Guidance
- Voice guidance
- Photo capture

Internet can optionally be used later for:

- Cloud backup
- Account synchronization
- AI-generated poses
- Online pose library
- Reference processing services if ever required

PROJECT STRUCTURE

Create clean, maintainable architecture.

Suggested structure:

```
app/ ├── camera/ ├── pose/ | ├── detection/ | ├── landmarks/ | ├── analysis/ | ├──  
matching/ | ├── rules/ ├── targetpose/ ├── guidance/ ├── audio/ ├── gallery/ ├──  
photoanalysis/ ├── ui/ | ├── home/ | ├── camera/ | ├── poses/ | ├── reference/ | ├──  
gallery/ | ├── settings/ ├── data/ ├── models/ ├── utilities/
```

Follow clean architecture and separation of concerns.

DATA MODELS

Create suitable models for:

User settings PoseTemplate Landmark JointAngle PoseComparison PoseCorrection GuidanceInstruction
PhotoAnalysis CameraState

Use strong typing.

Avoid hardcoding everything directly into composables/UI.

SETTINGS

Add settings such as:

- Voice guidance ON/OFF
- Show skeleton ON/OFF
- Show target pose ON/OFF
- Show match percentage ON/OFF
- Auto capture ON/OFF
- Countdown duration
- Instruction frequency
- Camera mirror mode

AUTO CAPTURE

Optional mode:

When pose score remains above a threshold for a configurable amount of time:

"Pose locked ✓"

Then:

3 2 1

Automatically capture.

Make auto capture configurable.

REFERENCE PHOTO PROCESSING

For reference images:

1. Load image.
2. Detect person.
3. Extract landmarks.
4. Normalize body coordinates.
5. Calculate angles.
6. Generate a target pose.
7. Save target pose representation.
8. Open camera.

9. Compare live pose with target pose.
10. Generate corrections.

If the reference image contains multiple people, initially support one main subject and clearly indicate that multi-person reference matching is an advanced feature.

BODY-SIZE NORMALIZATION

Do NOT compare raw pixel positions directly.

Normalize the pose using body dimensions.

Use a reference distance such as:

Shoulder width or Torso length

This allows a short person and tall person to match the same pose.

CAMERA POSITION / FRAME POSITION

Analyze the person's location in the camera frame.

For example:

If person center is too far left:

"Move right →"

If person center is too far right:

"Move left ←"

If person is too close:

"Move backward"

If person is too far:

"Move closer"

If head is outside safe framing:

"Move slightly down"

GROUP MODE

Use multi-person pose detection if supported.

Determine:

- Number of people
- Relative positions
- Spacing
- Individual pose correctness

For each person, display a small identifier:

Person 1 Person 2 Person 3

But keep the UI clean.

ERROR HANDLING

Handle:

- Camera permission denied
- No person detected
- Multiple people unexpectedly detected
- Poor lighting
- Person partially outside frame
- Unsupported device
- Pose detection unavailable
- Gallery permission denied
- Invalid reference photo
- No human detected in reference
- Too many people in reference

Messages should be user-friendly.

Examples:

"No person detected. Step into the frame."

"Move further away so your full body is visible."

"Try better lighting."

"Only one person should be visible for this pose."

DESIGN PRINCIPLES

Do not make the application overly complicated.

The main interaction should be:

Open → Point → Choose Pose → Follow Guidance → Capture

Use large readable instructions.

Avoid unnecessary menus.

Camera preview must dominate the screen.

Use smooth transitions.

Use modern cards and controls.

Use subtle animations.

MVP LIMITATION

IMPORTANT:

Do NOT try to implement everything at once.

Build the MVP first.

MVP must include:

1. Camera preview

2. Camera permission
3. Person detection
4. Pose landmarks
5. Skeleton overlay
6. 5–10 target poses
7. Pose comparison
8. Basic correction rules
9. Directional arrows
10. Text instructions
11. Match score
12. Capture photo
13. Basic photo quality check
14. Voice guidance
15. Reference image mode

After the MVP is stable, add advanced capabilities.

INITIAL POSES

Create at least these example target poses:

1. Neutral standing
2. Hands in pockets
3. One hand raised
4. Crossed arms
5. Casual side pose
6. Looking away
7. Leaning pose
8. One-leg-forward pose
9. Hand-on-waist pose
10. Simple portrait pose

Use sensible landmark/angle definitions.

Do not fake pose calculations.

DEVELOPMENT STRATEGY

Work in the following order.

PHASE 1: Create project and architecture.

PHASE 2: Build camera preview.

PHASE 3: Integrate pose detection.

PHASE 4: Display landmarks/skeleton.

PHASE 5: Implement landmark geometry and joint-angle calculations.

PHASE 6: Implement PoseTemplate model.

PHASE 7: Implement pose comparison.

PHASE 8: Implement correction engine.

PHASE 9: Implement visual arrows and overlay.

PHASE 10: Implement pose library.

PHASE 11: Implement reference image pose extraction.

PHASE 12: Implement text-to-speech guidance.

PHASE 13: Implement capture and gallery.

PHASE 14: Implement photo quality analysis.

PHASE 15: Performance optimization.

PHASE 16: UI polish.

PHASE 17: Testing.

TESTING

Create tests for:

- Joint angle calculations
- Landmark normalization
- Pose matching
- Rule engine
- Score calculation
- Correction priority

- Reference pose extraction
- Frame position detection

Also perform manual tests for:

- Different body heights
- Different camera distances
- Different lighting
- Different orientations
- One person
- Multiple people
- Partial body visibility
- Landscape
- Portrait

DOCUMENTATION

Create:

README.md

Include:

- Project overview
- Features
- Architecture
- Technology stack
- How pose detection works
- How pose matching works
- How to run the project
- Required permissions
- Build instructions
- Testing instructions
- Known limitations
- Future improvements

Also create:

ARCHITECTURE.md

and:

TODO.md

FUTURE FEATURES

Design the architecture so these can be added later:

1. AI-generated pose suggestions
2. Natural-language pose requests
3. "Give me a college Instagram pose"
4. "Give me a wedding pose"
5. "Give me a couple pose"
6. Pose recommendation based on location
7. Weather-aware outdoor suggestions
8. Clothing-aware pose suggestions
9. Automatic composition
10. Smart camera framing
11. AI background recommendations
12. Multiple reference images
13. Full body 3D pose guidance
14. AR ghost model
15. Pose history
16. Favorite poses
17. Social sharing
18. Cloud sync
19. User accounts
20. Personalized pose recommendations

Do not implement these unless the MVP is stable.

IMPORTANT CODING RULES

1. Write real working code.
2. Do not generate fake placeholder functionality when a real implementation is possible.
3. Do not create massive monolithic files.
4. Keep components modular.
5. Use meaningful names.
6. Add comments where the computer vision logic is difficult to understand.
7. Handle lifecycle correctly.
8. Handle camera permission correctly.
9. Avoid blocking the UI thread.
10. Optimize camera frame processing.
11. Avoid memory leaks.
12. Use proper state management.
13. Make the UI responsive.

14. Build for real physical devices, not only an emulator.
 15. Prefer on-device processing.
 16. Keep dependencies reasonable and maintained.
 17. Verify APIs against current documentation before implementation.
 18. If a library/API is unavailable or incompatible, choose the closest stable alternative.
 19. Never pretend a feature is implemented when it is only mocked.
 20. After each major phase, test the application and fix errors before continuing.
-

CLAUDE CODE WORKFLOW

You are operating inside a coding environment.

First inspect the existing directory.

Determine whether a project already exists.

If no project exists, initialize the Android project.

Do not overwrite existing user code unless absolutely necessary.

After inspecting the repository:

1. Explain the existing structure briefly.
2. Create the necessary architecture.
3. Implement the MVP in logical phases.
4. Run builds/tests frequently.
5. Fix compile errors immediately.
6. Verify dependencies.
7. Verify camera functionality.
8. Verify pose detection.
9. Verify pose matching.
10. Verify the main user flow.

Do not stop after generating files.

Actually compile/build the application.

When there is an error:

- Read the full error.
- Identify the root cause.
- Fix it.
- Rebuild.

- Repeat until the build succeeds.

PRIORITY OF FEATURES

The order of importance is:

1. Camera
2. Pose Detection
3. Pose Matching
4. Pose Correction
5. Visual Guidance
6. Capture
7. Reference Pose
8. Voice Guidance
9. Pose Library
10. Photo Quality Analysis
11. Group Mode
12. Advanced AI

Never sacrifice the core live pose-coaching experience to build secondary features.

FINAL USER EXPERIENCE

The completed MVP should allow the user to do this:

STEP 1:

Open PosePilot.

STEP 2:

Tap:

"AI POSE COACH"

STEP 3:

Camera opens.

STEP 4:

Choose:

"Hands in Pocket"

STEP 5:

Person stands in front of camera.

The app detects the person.

STEP 6:

The target pose appears as a ghost skeleton.

STEP 7:

The app says:

"Move your right hand into your pocket."

The user moves.

STEP 8:

The app says:

"Turn your body slightly left."

The user moves.

STEP 9:

The app says:

"Look at the camera."

STEP 10:

Pose reaches:

96%

STEP 11:

The app says:

"Perfect. Hold."

STEP 12:

3...2...1...

STEP 13:

Photo captured.

STEP 14:

Photo quality analysis appears:

Pose ✓ Framing ✓ Face ✓ Blur ✓

STEP 15:

The photo can be saved.

PRODUCT POSITIONING

Use this conceptual description inside the project README:

"PosePilot is an AI-powered real-time photography assistant that uses computer vision and pose estimation to guide people into better photographic poses. Instead of simply showing pose references, PosePilot actively analyzes the user's body through the camera and provides visual, textual and voice instructions to help them achieve the desired pose."

IMPORTANT

The project should look and behave like a real product prototype, not a simple academic demo.

Prioritize:

- Real-time performance
- Accurate pose guidance
- Simple UX
- Clean architecture

- Expandability
- On-device processing
- Privacy
- Good visual design

Start by inspecting the current repository and then implement the project step by step.

Do not wait for me to manually provide every file.

Make reasonable engineering decisions yourself.

When a decision has multiple valid options, choose the option that is simplest, stable, maintainable, and appropriate for an Android MVP.

Begin now.